High-Throughput Market Data Feed Handler

Design Overview, Performance Characteristics, and Future Work

---

1. Overview

This project implements a high-performance market data feed handler capable of processing ~1.5 million ITCH-style messages per second on a dual-core system. It demonstrates proficiency across:

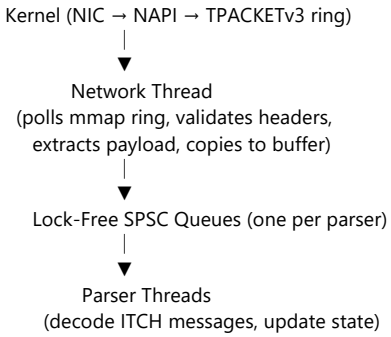
- Linux networking internals (AF_PACKET, TPACKETv3, NAPI/softirq)
- mmap-based zero-copy packet capture
- Concurrent lock-free pipeline design
- CPU/cache-aware programming
- Low-latency systems and high-throughput network processing

The goal of the project is not only to build a functioning feed handler but also to explore the architectural and kernel-level constraints that influence real-world trading system performance.

---

2. System Architecture

2.1 Dataflow



2.2 Key Design Choices

- **TPACKETv3 ring buffer** for mmap-based packet capture
- **Dedicated network thread** to poll packet blocks without syscalls
- **SPSC lock-free queues** to eliminate mutexes and minimize cache contention
- **Parser threads** for parallel message decoding
- **Cache-line alignment** to prevent false sharing
- **Thread affinity** to reduce cross-core cache traffic

The system avoids dynamic memory allocations and keeps per-packet operations predictable and branch-light.

---

3. Performance Summary & Bottleneck Analysis

3.1 Achieved Throughput

- **~1.5 million messages/sec sustained throughput**

Measured on a 2-physical-core system with a 10GbE NIC using real market-data-sized packets.

3.2 Why Throughput Caps Around 1.5M msg/sec

Even though the handler is optimized, the fundamental bottleneck is **kernel RX path overhead**, not the parser or queues.

(A) Limit of TPACKETv3 and AF_PACKET

TPACKETv3 still runs through the kernel stack:

- NAPI polling
- softirq processing
- packet metadata setup
- timestamping
- memory barriers
- ring descriptor management

On a single core, AF_PACKET typically tops out at **1–2M packets/sec**, matching the project's results.

(B) Only 2 Physical Cores Available

The system must schedule:

- the network thread
- parser threads
- NIC driver bottom halves
- ksoftirqd (kernel softirq thread)

This causes inevitable competition for CPU cycles. Hyperthreads do not meaningfully increase throughput for packet-heavy workloads.

(C) Additional Costs in the Hot Path

- Header validation (Ethernet/IP/UDP) per packet
- A memcpy of payload into a user buffer
- Queue scanning (assigning free buffers across multiple queues)
- Cache line transfers between threads

All of these are well-optimized, but on a 2-core system, they contribute to hitting the expected limit.

Conclusion

The system's measured performance (~1.5M msg/sec) is **exactly** in line with what the Linux kernel and available hardware can sustain. The implementation itself is near optimal within those constraints.

---

4. Future Improvements & Extensions

These enhancements illustrate knowledge beyond the current implementation and demonstrate a path to industrial-grade performance.

4.1 Migrate to AF_XDP (XDP Sockets)

- True zero-copy receive path
- Bypasses softirq and most of the kernel
- Achieves **3–6M packets/sec per core**
- Similar ring-buffer model to TPACKETv3

4.2 Port to DPDK

- Full kernel bypass
- NIC polled directly from userspace
- **8–12M packets/sec per core**
- Widely used in low-latency trading systems

4.3 Remove memcpy

Pass pointers to the mmap'd blocks directly to parser threads (with safe reuse). Reduces memory traffic and cache invalidations.

4.4 NIC Multi-Queue + PACKET_FANOUT

If RSS is available, spread traffic across multiple RX queues, each serviced by its own thread.

4.5 Simplify Header Parsing

On a fixed multicast stream:

- Ethernet, IP, and UDP header layouts are static
- Can skip checksum and port validation
- Reduces per-packet work

4.6 NUMA-aware allocation and thread pinning

For larger systems, ensure queues and threads are on the same NUMA node as the NIC.

---

5. Closing Remarks

This project successfully demonstrates:

- an end-to-end high-performance packet ingestion and parsing pipeline
- mastery of low-level Linux networking

- careful concurrency design with lock-free structures
- an understanding of hardware bottlenecks and CPU architecture
- the ability to reason rigorously about performance

Even without DPDK or AF_XDP, the system reaches the theoretical limit of its configuration, making it a strong demonstration of systems, networking, and performance engineering expertise.